# Random Walks: From Mathematical Foundations to Modern Applications in Artificial Intelligence

Kam Seng Lei
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, China

Kaige Liang
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, China

***Abstract*—The random walk is one of the most influential mathematical objects in probability theory and a foundational primitive throughout modern artificial intelligence (AI). In this report we trace the concept from its simplest incarnation — a one-dimensional walk driven by independent coin flips — to its quantitative consequences (linear drift, $\sqrt{n}$ diffusion, and the central limit theorem) and to its dimensional behaviour as captured by Pólya's recurrence theorem. We then survey how the random-walk abstraction is used inside contemporary AI: it underpins Google's PageRank, the noising/denoising mechanism of diffusion-based generative models, random-walk graph embeddings such as DeepWalk and node2vec, Markov chain Monte Carlo (MCMC) inference, and exploration in reinforcement learning. The report also documents how we prepared the class demonstration: each formula is paired with a visual simulation or AI example so that the concept is both mathematically precise and easy to explain. The accompanying GitHub Pages supplement is available at https://kaigeliang.github.io/random-walks-ai-report/.**

***Index Terms*—random walk, central limit theorem, Markov chain, Pólya's theorem, PageRank, diffusion model, graph embedding, MCMC, reinforcement learning**

## I. Introduction

Stock prices, the trajectory of a pollen grain in water, the path of a web surfer clicking from page to page, the noise schedule of a modern image generator — on the surface, these phenomena have very little in common. What unites them is a deceptively simple mathematical object: the *random walk*.

A random walk is a path built from a sequence of random steps. At every moment, "chance" decides where the walker goes next, independently of where it has been. Repeat that rule many times, and the walker traces out a jagged, erratic trajectory whose statistical regularities are remarkably deep [1], [2]. The randomness at the level of a single step gives rise to surprisingly rigid order at the level of the ensemble: a linear average drift, a spread that grows like the square root of time, and — in the limit — the universal bell curve of the central limit theorem (CLT).

Historically the random walk has a remarkable lineage. The term was coined by Karl Pearson in a 1905 letter to *Nature* in which he asked how far a drunk man would have wandered after $n$ steps. Five years earlier, Louis Bachelier had used essentially the same model to study fluctuations on the Paris stock exchange in his Sorbonne thesis, and in 1905 Albert Einstein independently invoked it to predict the motion of particles suspended in a fluid — a result that Jean Perrin later turned into experimental evidence for the existence of atoms. The same object underpinned Andrey Markov's analysis of letter dependencies in *Eugene Onegin*, and reappeared in George Pólya's celebrated 1921 recurrence theorem. A century on, it remains a load-bearing primitive in machine learning, network science, and statistical physics.

This report has three goals. First, in Section II we document how we prepared the demonstration and then lay out the probabilistic background needed to make the walk precise. Second, in Sections III and IV we develop the mathematics of the one-dimensional walk and its higher-dimensional extensions, recovering the results presented in our class talk. Third, and most importantly, Section V explains how the random-walk abstraction is reused across modern AI: ranking, generation, embedding, inference, and exploration. Section VI concludes.

## II. Preparation

Our project was prepared as a two-layer demonstration. The first layer builds intuition from one fair coin walk: we show individual sample paths, the $\sqrt{n}$ envelope, the CLT histogram, and the recurrence contrast in low versus high dimensions. The second layer reuses the same transition rule inside AI systems, so the audience can see that PageRank, diffusion models, graph embeddings, MCMC, and RL exploration are not isolated examples but different algorithmic uses of the same random-walk idea. Kam Seng Lei led the mathematical storyline, including the definitions, drift and variance derivations, CLT explanation, Pólya theorem summary, and reference checking. Kaige Liang led the computational and AI storyline, including the Python simulations, figures, PageRank and diffusion examples, and the mapping from each equation to a machine learning application. Both members rehearsed the demonstration together, cross-checked equations and captions, and edited the final report for clarity and consistency.

To make the static report easier to present and reproduce, we also built a lightweight interactive supplement for GitHub Pages. The page is stored in `docs/index.html` in the project repository and animates the same five AI examples discussed in Section V: the PageRank random surfer, forward diffusion noising, DeepWalk-style graph sampling, random-walk Metropolis inference, and $\epsilon$-greedy exploration in reinforcement learning. This supplement is not a separate model or

experimental claim; it is a visual companion to the equations and figures used in the report. The live version is available at https://kaigeliang.github.io/random-walks-ai-report/.

We model uncertainty on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ in the standard Kolmogorov sense [3]. A real-valued *random variable* $X : \Omega \to \mathbb{R}$ has expectation $\mathbb{E}[X] = \int X \, d\mathbb{P}$ and variance $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$. Two key properties drive everything that follows.

1) **Linearity of expectation.** For any random variables $X_1, \ldots, X_n$ and constants $a_i$,

$$\mathbb{E}\left[\sum_{i=1}^{n} a_i X_i\right] = \sum_{i=1}^{n} a_i \, \mathbb{E}[X_i], \tag{1}$$

regardless of whether the $X_i$ are independent.

2) **Variance of independent sums.** If $X_1, \ldots, X_n$ are *independent*, then

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i). \tag{2}$$

Independence is essential here: cross-terms vanish only because the covariances are zero.

A sequence $X_1, X_2, \ldots$ that is *independent and identically distributed* (i.i.d.) is the basic building block of a random walk: every step is drawn from the same distribution, and no step has any memory of the previous ones. The discrete-time stochastic process $\{S_n\}_{n \geq 0}$ obtained by partial summation of i.i.d. steps is itself a Markov chain on the integers (or on $\mathbb{Z}^d$, or on a graph), since the next state depends on the past only through the present. This *Markov property* is the hinge that connects elementary random walks to the broader theory of stochastic processes:

$$\begin{aligned}\mathbb{P}\big(S_{n+1} = y \mid S_n = x, \, S_{n-1}, \ldots, S_0\big) \\ = \mathbb{P}\big(S_{n+1} = y \mid S_n = x\big).\end{aligned} \tag{3}$$

Many of the AI applications surveyed in Section V ultimately reduce to algorithmic exploitation of this single equation.

## III. The One-Dimensional Random Walk

### A. Setup

Place a walker at the origin. At every discrete time step the walker takes a step of size one to the right or to the left, with probabilities $p$ and $1 - p$ respectively. Formally, let

$$X_i = \begin{cases} +1 & \text{with probability } p, \\ -1 & \text{with probability } 1 - p, \end{cases} \tag{4}$$

i.i.d. across $i$. The position after $n$ steps is the random variable

$$S_n = \sum_{i=1}^{n} X_i, \qquad S_0 = 0. \tag{5}$$

### B. Average behaviour: linear drift

Equation (4) gives

$$\mathbb{E}[X_i] = p \cdot (+1) + (1 - p) \cdot (-1) = 2p - 1. \tag{6}$$

By linearity (1),

$$\mathbb{E}[S_n] = n(2p - 1). \tag{7}$$

For a fair coin ($p = \frac{1}{2}$) the walk is unbiased: $\mathbb{E}[S_n] = 0$ for all $n$. A small bias produces a sizeable drift over many steps: $p = 0.51$ and $n = 10{,}000$ already give $\mathbb{E}[S_n] = 200$.

### C. Spread: the $\sqrt{n}$ envelope

The single-step variance is

$$\mathrm{Var}(X_i) = \mathbb{E}[X_i^2] - \mathbb{E}[X_i]^2 = 1 - (2p-1)^2 = 4p(1-p). \tag{8}$$

Because the steps are independent, (2) immediately gives

$$\mathrm{Var}(S_n) = n\big[1 - (2p - 1)^2\big] = 4np(1 - p), \tag{9}$$

so the standard deviation scales like $\sqrt{n}$. For a fair coin $\mathrm{Var}(S_n) = n$ and $\sigma(S_n) = \sqrt{n}$.

Fig. 1. Eight independent fair walks of length 400. The deterministic $\pm\sqrt{n}$ envelope (gold) tracks the typical magnitude of $S_n$: the cloud widens, but only sub-linearly.

The qualitative consequence is striking: ten times more steps only widens the typical distance from the origin by a factor of $\sqrt{10} \approx 3.16$. After 100 fair steps the walker is typically about 10 units from the start, not 50. Figure 1 shows eight independent walks together with the $\pm\sqrt{n}$ envelope.

### D. Central limit theorem

The most striking regularity emerges when we rescale. Define

$$Z_n = \frac{S_n - n(2p - 1)}{\sqrt{n\big[1 - (2p - 1)^2\big]}}. \tag{10}$$

The classical Lindeberg–Lévy CLT [3] states that $Z_n \xrightarrow{d} \mathcal{N}(0,1)$ as $n \to \infty$, where the arrow denotes convergence in distribution. For the symmetric coin walk this simplifies to

$$\frac{S_n}{\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1). \tag{11}$$

Empirically, the histogram of the rescaled walk converges to the standard Gaussian as $n$ grows; see Fig. 2. This is the universality result: regardless of the fine details of the step distribution (as long as it has finite variance), the macroscopic shape of the walk is a bell curve.

### E. Diffusive scaling and Brownian motion

The $\sqrt{n}$ growth and the Gaussian limit are two faces of the same phenomenon: the random walk is a discrete approximation of *Brownian motion* [4]. If we shrink the spatial step to $\Delta x = \sqrt{\Delta t}$ and the time step to $\Delta t \to 0$, the rescaled walk converges (in the sense of Donsker's invariance principle) to standard Brownian motion $B_t$, a continuous-time process with independent, Gaussian increments. The same diffusive scaling will reappear later in this report when we discuss generative diffusion models.

## IV. Higher Dimensions and Pólya's Theorem

The simple random walk generalises naturally to the integer lattice $\mathbb{Z}^d$. At each step the walker chooses one of the $2d$ axial neighbours uniformly at random. The expectation and variance arguments of Section III carry over coordinate-wise; the $\sqrt{n}$ scaling is unchanged.

What changes dramatically with dimension is the question of *recurrence*.

**Definition 1.** *A random walk is* recurrent *if it returns to its starting point with probability one, and* transient *otherwise.*

**Theorem 1** (Pólya, 1921 [5])**.** *The simple symmetric random walk on $\mathbb{Z}^d$ is recurrent for $d \in \{1,2\}$ and transient for $d \geq 3$.*

The contrast is famously summarised by Kakutani's quip: *a drunk man will eventually find his way home, but a drunk bird may get lost forever*. The proof inspects the expected number of returns to the origin,

$$\mathbb{E}\left[\sum_{n=0}^{\infty} \mathbf{1}\{S_n = 0\}\right] = \sum_{n=0}^{\infty} \mathbb{P}(S_n = 0), \tag{12}$$

and uses the local CLT to show $\mathbb{P}(S_n = 0) \sim Cn^{-d/2}$. The series diverges for $d \leq 2$ (forcing recurrence) and converges for $d \geq 3$ (allowing escape).

This dimensional dichotomy is far from a curiosity. It says something deep about the geometry of randomness: in low dimensions there is "not enough room" to escape, so paths self-intersect repeatedly, while in high dimensions there is "too much room", and a typical path escapes to infinity. Both regimes are exploited in modern AI — often, remarkably, in the same algorithm.

## V. Applications in Artificial Intelligence

The random walk is not merely a mathematical curiosity — it is a working primitive inside several pillars of modern AI. We organise the discussion around five representative applications.

### A. PageRank and link-based ranking

Google's original ranking algorithm, PageRank [6], models a hypothetical *random surfer*: a user that, at each step, either clicks a uniformly chosen link on the current page (with probability $1-\alpha$) or teleports to a uniformly chosen page (with probability $\alpha \approx 0.15$). This defines an irreducible, aperiodic Markov chain on the web graph. The PageRank score of a page is the stationary probability that the surfer occupies that page in the long run.

Figure 3 illustrates the idea on a five-page toy graph: the gold node is the one a random surfer occupies most often in the long run, even though it does not have the highest raw in-degree. Concretely, if $A$ is the column-stochastic transition matrix and $\mathbf{e}$ the uniform vector, the PageRank vector $\mathbf{r}$ satisfies

$$\mathbf{r} = (1-\alpha)A\mathbf{r} + \alpha\mathbf{e}, \tag{13}$$

which is solved efficiently by power iteration. The teleportation term guarantees recurrence — exactly the property Pólya's theorem formalises — and therefore a unique stationary distribution. Link-prediction systems, citation analysis, recommendation engines, and fraud-detection pipelines all extend or specialise this random-walk formulation [7]. *Personalised* PageRank, which biases the teleportation distribution toward a query node, is particularly popular in recommender systems: it captures the intuition that pages frequently visited from a seed are likely relevant to a user who has just viewed that seed.

### B. Diffusion models for generative AI

Modern image, audio, and video generators (Stable Diffusion, Imagen, Sora, ...) are built around *denoising diffusion probabilistic models* (DDPMs) [8]. Their forward process is, quite literally, a random walk in pixel space: starting from a clean datum $x_0$, one adds small Gaussian increments,

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I), \tag{14}$$

until $x_T$ is indistinguishable from pure noise. The continuous-time limit is exactly the Brownian motion described in Section III [9]. Figure 4 traces a one-dimensional toy example: a clearly bimodal data distribution at $t = 0$ is gradually smeared by Gaussian increments until, at $t = T$, it is statistically indistinguishable from $\mathcal{N}(0,1)$. Generation is the time *reverse* of this walk: a neural network is trained to estimate the score $\nabla_x \log p_t(x)$ at every noise level, and a numerical SDE solver runs the diffusion backwards from $x_T$ to $x_0$. The $\sqrt{n}$ spreading we proved in Eq. (9) is the very reason the noising schedule must be square-root in time; the CLT (11) is the reason the limiting distribution is tractable Gaussian.

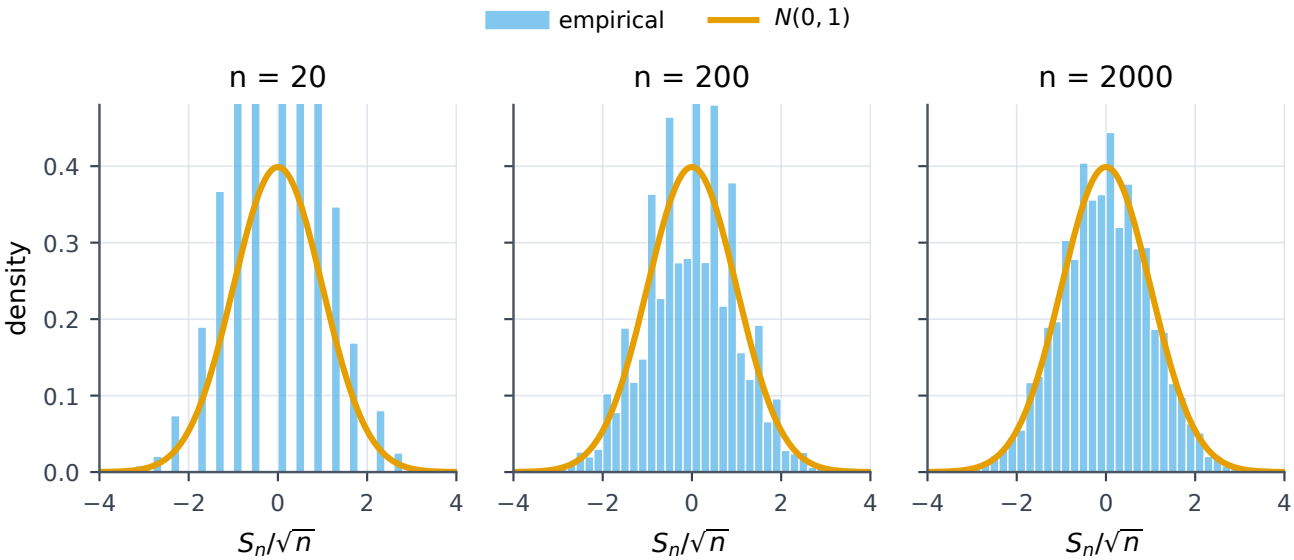


Fig. 2. Empirical density of $S_n/\sqrt{n}$ for a fair walk with $N = 12{,}000$ replicates and $n \in \{20, 200, 2000\}$. The gold curve is the standard normal $\mathcal{N}(0, 1)$. Larger $n$ produces a closer match between the histogram and the limiting Gaussian.

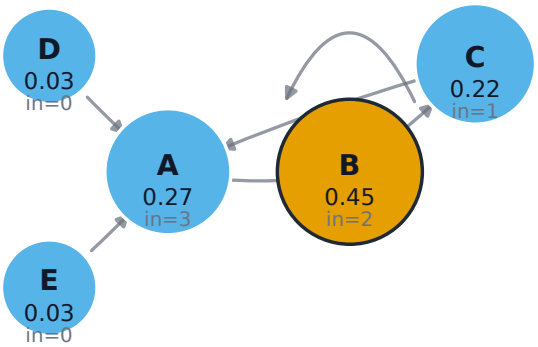


Fig. 3. Toy web graph with five pages. The stationary distribution of the random-surfer Markov chain assigns the highest probability mass to the most "trafficked" node (gold), regardless of its raw in-degree.

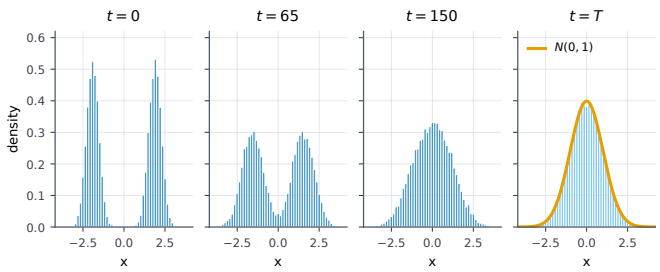


Fig. 4. Forward diffusion on a one-dimensional toy distribution. A clean bimodal $x_0$ (left) is gradually noised by a Gaussian random walk until $x_T$ matches the standard normal (gold reference). Generation reverses this walk via a learned score model.

### C. Random-walk graph embeddings: DeepWalk and node2vec

Representing graph nodes as low-dimensional vectors is a central problem in network analysis. DeepWalk [10] treats short random walks on a graph as "sentences" and feeds them to a Skip-gram language model: nodes that co-occur on the same walk receive similar embeddings (Fig. 5).

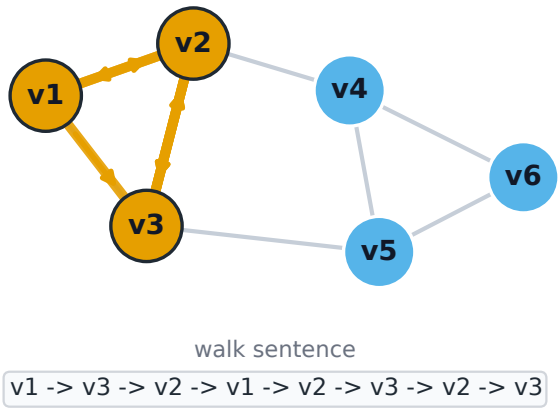


Fig. 5. A short random walk on a graph (gold path) plays the role of a "sentence" in DeepWalk and node2vec. The Skip-gram objective then pulls the embeddings of co-occurring nodes together, producing neighbourhood-aware vector representations.

The node2vec method [11] generalises this with a biased walk that interpolates between breadth-first (homophily) and depth-first (structural equivalence) exploration:

$$\pi(v \to x \mid t) = \begin{cases} 1/p & d_{tx} = 0, \\ 1 & d_{tx} = 1, \\ 1/q & d_{tx} = 2, \end{cases} \tag{15}$$

where $d_{tx}$ is the shortest-path distance from the previous node $t$ to the candidate $x$. These embeddings have been used to predict drug interactions, recommend products on Pinterest, classify proteins, and detect financial fraud — always exploiting the random walk as a neighbourhood sampler. Personalised PageRank, computed from short random walks rooted at a query node, plays the same role inside more recent graph neural networks [12].

### D. Markov chain Monte Carlo

Whenever a probability distribution $\pi(x)$ is known only up to a constant — as is typical for Bayesian posteriors and energy-based models — it can still be sampled by constructing a Markov chain whose stationary distribution is $\pi$. The Metropolis–Hastings random walk proposes $x' \sim q(x' \mid x_t)$, accepts with probability

$$\alpha(x_t, x') = \min\left\{1, \frac{\pi(x')\, q(x_t \mid x')}{\pi(x_t)\, q(x' \mid x_t)}\right\}, \tag{16}$$

and otherwise stays put [13]. Hamiltonian Monte Carlo (HMC) and Langevin dynamics use Brownian-motion proposals derived from the gradient of $\log \pi$. Figure 6 shows a small Metropolis chain exploring a banana-shaped target: a few dozen steps trace the random walker's path through the support, and after a few thousand the empirical distribution of the chain matches the contours of the target.

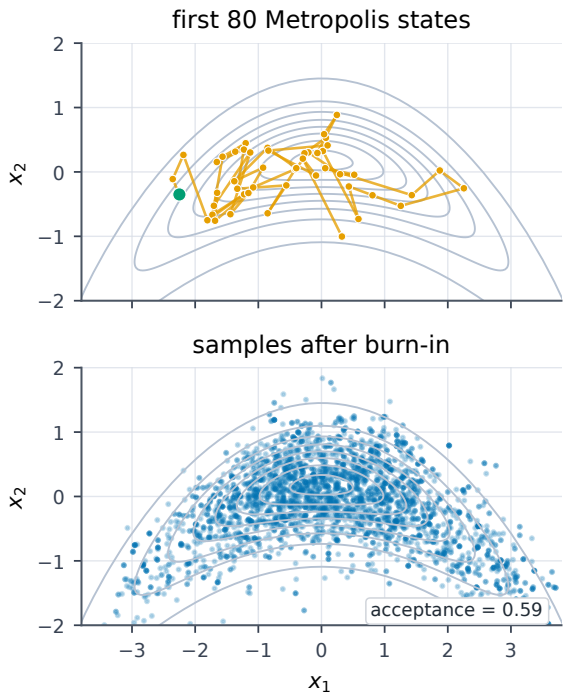


Fig. 6. Random-walk Metropolis on a two-dimensional banana-shaped target. **Top:** the first 80 samples (gold dots) traced by the proposal–accept random walk, overlaid on the target's level sets. **Bottom:** after 4000 steps the empirical distribution of the chain (navy dots) faithfully tracks the target. Mixing time controls how fast this match is achieved.

MCMC underpins probabilistic programming languages (Stan, PyMC), Bayesian deep learning, posterior inference in modern diffusion samplers, and the training of restricted Boltzmann machines. The convergence rate of these algorithms is exactly a random-walk mixing-time question [14]: poorly chosen proposal kernels diffuse too slowly to escape a local mode, while overly aggressive ones reject most moves. Tuning these random walks — adaptive step sizes, parallel tempering, gradient-informed proposals — is one of the most active areas in computational statistics.

### E. Exploration in reinforcement learning

Reinforcement-learning (RL) agents must *explore* an unknown environment before they can exploit it. A classical baseline is the $\epsilon$-greedy policy that chooses a random action with probability $\epsilon$ — a random walk on the action space. More sophisticated strategies still rely on the random-walk abstraction: option-critic methods discover sub-policies via random walks on the state graph, successor-feature methods compute occupancy measures that are exactly the visit frequencies of a random walk, and "go-explore" style algorithms [15] use random walks from previously visited cells to discover novel states. The exploration-exploitation trade-off can be expressed as a random walk with a learnt bias — the RL analogue of choosing the parameter $p$ in Eq. (4).

### F. A unifying view

Across these five families a common pattern emerges. The forward random walk produces samples from a target stationary distribution (PageRank, MCMC), or it noises a signal in a controlled way (diffusion models). The walk's structural properties — linear drift, $\sqrt{n}$ spread, the Gaussian limit of Eq. (11), and the recurrence/transience dichotomy of Pólya's theorem — determine whether the algorithm mixes, converges, generalises, or escapes. Far from being an introductory example in a probability course, the random walk is a load-bearing abstraction inside the systems we use every day.

## VI. Conclusion

We started from a coin: heads we step right, tails we step left. With nothing more than the linearity of expectation and the additivity of variance over independent summands, we derived the linear drift $\mathbb{E}[S_n] = n(2p-1)$ and the diffusive spread $\sigma(S_n) = \sqrt{4np(1-p)}$. Taking the limit gave the central limit theorem; lifting to higher dimensions exposed Pólya's recurrence dichotomy. We then traced the same abstraction through five active research areas in AI: PageRank, diffusion-based generative models, random-walk graph embeddings, MCMC inference, and exploration in reinforcement learning. The lesson of the report is that elementary probability, properly understood, scales: the same coin flip we analysed in Section III powers the algorithms behind today's most sophisticated AI systems.

## REFERENCES


[1] F. Spitzer, *Principles of Random Walk*, 2nd ed. New York, NY, USA: Springer, 2001.

[2] G. F. Lawler and V. Limic, *Random Walk: A Modern Introduction*. Cambridge, U.K.: Cambridge Univ. Press, 2010.

[3] R. Durrett, *Probability: Theory and Examples*, 5th ed. Cambridge, U.K.: Cambridge Univ. Press, 2019.

[4] P. Mörters and Y. Peres, *Brownian Motion*. Cambridge, U.K.: Cambridge Univ. Press, 2010.

[5] G. Pólya, "Über eine Aufgabe der Wahrscheinlichkeitsrechnung betreffend die Irrfahrt im Straßennetz," *Mathematische Annalen*, vol. 84, no. 1–2, pp. 149–160, 1921.

[6] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999–66, 1999.

[7] D. F. Gleich, "PageRank beyond the web," *SIAM Review*, vol. 57, no. 3, pp. 321–363, 2015.

[8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NeurIPS*, vol. 33, 2020, pp. 6840–6851.

[9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. ICLR*, 2021.

[10] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. ACM SIGKDD*, 2014, pp. 701–710.

[11] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. ACM SIGKDD*, 2016, pp. 855–864.

[12] J. Klicpera, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized PageRank," in *Proc. ICLR*, 2019.

[13] W. K. Hastings, "Monte Carlo sampling methods using Markov chains and their applications," *Biometrika*, vol. 57, no. 1, pp. 97–109, 1970.

[14] D. A. Levin and Y. Peres, *Markov Chains and Mixing Times*, 2nd ed. Providence, RI, USA: AMS, 2017.

[15] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune, "First return, then explore," *Nature*, vol. 590, no. 7847, pp. 580–586, 2021.